Auteurs

Leseigneur Bastien

bastien.leseigneur@imt-atlantique.net

Diers Théodore

theodore.diers@imt-atlantique.net

Perrin Lucas

lucas.perrin@imt-atlantique.net

Rapport PROJET 28 CODEVSI

Détection des communications AIS par satellite en présence d'interférences d'accès multiple

Date : 15/05/2024

Formation/année : FISE A1

Destinataires : Karine Amis, Ahmed Masmoudi et le Comité de Pilotage du projet CODEVSI

Version 2.0

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Résumé

Rédacteur: Bastien Leseigneur, Relecteurs: Théodore Diers, Lucas Perrin

Le système Automatic Identification System (AIS) est un système de communication permettant plus de sécurité en mer, notamment en limitant le risque de collisions entre navires. Bien qu'efficace, son fonctionnement possède une lacune, les informations relayées par ce système de communication ne le sont que sur un petit rayon autour du bateau émetteur, et donc ne sont pas directement reçues par les stations de contrôle situées sur le continent. Il y a quelques années, des constellations de satellites ont été déployées en orbite basse autour de la Terre afin de pouvoir recueillir instantanément ces signaux AIS émis par les navires présents en mer. Néanmoins, l'utilisation de ces satellites a grandement augmenté les probabilités de collisions entre messages envoyés par différents bateaux, créant donc un risque non négligeable de perte d'informations.
Notre projet répond à un besoin des chercheurs du département MEE en collaboration avec le CNES, Kinéis et le laboratoire TéSA qui souhaitent étudier ces collisions entre messages. Nous avons été chargés de concevoir un simulateur de communications AIS entre un terminal situé sur un navire et un de ces satellites. Plus précisément, nous avons généré un canal de transmission entre une position aléatoire d'un terminal et un satellite, avec tous les paramètres correspondant. Nous avons aussi généré un signal AIS respectant le format imposé et construit des fonctions permettant de modéliser la modulation du signal, sa transmission à travers le canal, ainsi que l'estimation du signal reçu par le satellite. Nous n'avons toutefois pas eu le temps de mettre en place l'estimation du nombre de navires en présence et des paramètres retard-décalage Doppler ni l'étude des performances de la détection des signaux AIS.
Ce projet a été conçu pour évoluer par la suite en fonction des besoins des chercheurs du département MEE.

Table des matières

Résumé....2
Table des matières....3
I - Introduction....4
II - Simulation de l'environnement....6
II.1 - Génération aléatoire de la position d'un terminal....6
II.2 - Détermination des paramètres caractérisant le lien terminal satellite....9
III - Génération du signal....12
III.1 - Génération aléatoire des bits....13
III.2 - Codage CRC....13
III.3 - Bitstuffing....14
III.4 - Flags....14
III.5 - Training sequence....15
III.6 - Codage NRZI....15
III.7 - Modulation GMSK....16
IV - Réception et décodage....18
IV.1 - Estimation du signal reçu....18
IV.2 - Algorithme de Viterbi....19
IV.3 - Décodage du signal....19
IV.4 - Performances du détecteur....20
V - Conclusion....20
VI - Glossaire....21
VII - Annexes....22
VII.1 - Comparaison des plannings réels et prévisionnels....23
VIII - Bibliographie....27

I - Introduction

Rédacteurs: Bastien Leseigneur, Lucas Perrin, Relecteur: Théodore Diers

Le système Automatic Identification System (AIS) est un système de communication permettant de limiter le risque de collisions entre navires et d'améliorer la sécurité maritime. Les bateaux sont équipés d'émetteurs-récepteurs radio qui émettent de manière régulière un signal contenant des informations importantes sur celui-ci, comme son cap, sa vitesse, sa position, etc. Les navires présents dans le rayon de portée de cet émetteur reçoivent ces informations et s'adaptent en conséquence. Les océans sont divisés en cellules de quelques miles nautiques afin que chaque bateau présent dans cette cellule puisse communiquer, c'est-à-dire émettre et recevoir les signaux AIS, avec les navires et stations côtières présents dans cette cellule. Les émetteurs-récepteurs partagent la bande passante radio allouée au fonctionnement AIS en utilisant des techniques d'accès multiples par répartition dans le temps (TDMA). Chaque minute, un intervalle de quelques millisecondes est réservé à un navire pour qu'il puisse transmettre ses informations. Étant donné qu'il y a plus de 2000 slots par minutes pour le système AIS, le risque d'interférences dans une même cellule est extrêmement faible.[4]

Afin que des centres de contrôle du trafic maritime soient mis en place, des balises ont été placées dans l'océan à des endroits stratégiques pour capter ces signaux. Mais cette méthode est limitée puisqu'elle ne permet pas de couvrir toute la surface de la Terre. Alors le système AIS s'est développé en utilisant une constellation de satellites en orbite basse chargée de réceptionner les signaux AIS de chaque bateau autour de la Terre.
Toutefois ces satellites ont un angle de couverture à 3dB de 130°, ce qui, considérant le fait qu'ils soient situés à 656.5 km d'altitude, amène une surface couverte par ce satellite largement plus grande qu'une seule cellule AIS. Cela signifie que la probabilité que plusieurs signaux entrent en collision augmente drastiquement. Des techniques de réception plus ou moins sophistiquées sont alors nécessaires pour réduire la perte des messages.

L'objectif de ce projet est de réaliser un simulateur Matlab de communications entre un terminal situé sur Terre.

Nous avions comme objectifs de mettre en place la génération aléatoire d'un signal AIS suivant les normes imposées par le standard, avec notamment comme contrainte de devoir moduler le signal par modulation dite GMSK. Nous devions aussi réaliser la simulation de l'environnement en générant les paramètres de position du terminal, ainsi qu'en estimant sa distance au satellite, le décalage Doppler subi par le signal envoyé, le retard de propagation et le bilan de liaison entre le terminal et le satellite. Enfin, nous avions comme fonctionnalités principales la détection du signal AIS en sortie de canal et l'estimation des paramètres Doppler et retards par une méthode de corrélation.

Nous répondrons donc dans ce document à la question suivante: comment concevoir un tel simulateur avec le logiciel Matlab ?

Nous verrons donc dans un premier temps comment nous avons géré la simulation de l'environnement avec d'abord la génération aléatoire des positions du terminal dans la zone de couverture du satellite puis ensuite la détermination des paramètres caractéristiques du

lien Terminal-Satellite et la réalisation du bilan de liaison entre ces deux éléments. Une fois l'environnement de simulation construit, nous présenterons la génération du signal AIS respectant le format imposé avant de présenter la méthode utilisée pour coder notre séquence de bits afin de la moduler pour l'envoyer à travers le canal. Enfin nous parlerons de la phase de réception et de décodage du signal.

II - Simulation de l'environnement

Rédacteur: Bastien Leseigneur, Relecteurs: Théodore Diers, Lucas Perrin

II.1 - Génération aléatoire de la position d'un terminal

Pour simplifier le problème, nous supposons que le satellite est fixe au-dessus du pôle nord de la Terre. Son antenne est pointée à la verticale, dans la direction de la planète, et la regarde avec un angle d'ouverture à 3dB de $\theta_{3dB} = 130\,°$. La surface projetée visible par le satellite est alors une calotte de la forme suivante:

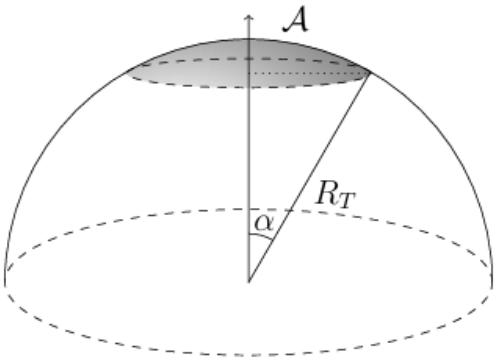


Figure 2.1 - Représentation de la calotte d'angle α et d'aire A visible par le satellite *[1]*

La première chose à réaliser est donc de générer la position d'un terminal à l'intérieur de cette zone. A noter que toutes les valeurs utilisées dans notre simulateur sont données dans l'annexe 1.

1. Première méthode

Soit une matrice carrée $M \in M_n(\Re)$. On prend généralement n = 1000 amenant à n² positions possibles du terminal. L'objectif est d'associer à chacun des coefficients $m_{i,j}$ de cette matrice une position sur la calotte. Pour cela, on utilise la bijection suivante:
$\varphi: \{ m_{i,j} \rightarrow r(i,\, j) \;=\; max\,(\,|i \;-\; n/2|,\, |j \;-\; n/2|)\; et\; \theta(i,j) \;=\; arctan((j \;-\; n/2\,)/(i \;-\; n/2))$
Ici r représente la distance du point à l'axe pôle nord-pôle sud du centre de la Terre et θ un angle par rapport au méridien de Greenwich, c'est-à-dire une longitude.

Il faut ensuite normaliser r pour que chacun des points de la grille soit visible par le satellite. Pour cela on introduit $r_{normalisé}$ qui vaut:
$r_{normalisé}(i,j) \;= r_{max} \cdot r(i,\, j)/r(1,\, 1)$ où $r_{max} \;=\; d_{max} \cdot sin(\theta_{3dB}/2)$ avec d_{max} la distance maximale entre le satellite et un terminal, distance que l'on peut obtenir par relation géométrique.

On convertit ensuite les couples $(r_{normalisé},\, \theta)$ en couples (latitude, longitude) par les relations suivantes:

$latitude = arccos(r_{normalisé}/R_T)$ avec R_T le rayon de la Terre

$longitude = \theta$

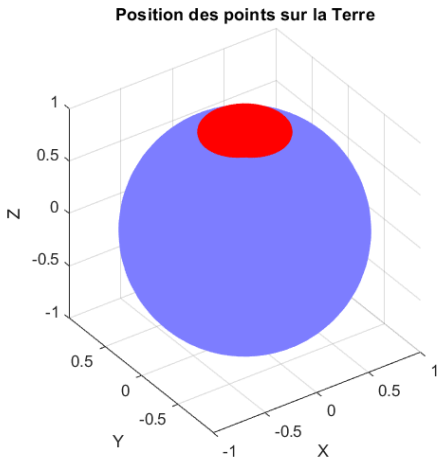


Figure 2.2 - Représentation normalisée de l'ensemble positions possibles du terminal donnée par la première méthode, chaque point rouge correspondant à une position possible.

Cette représentation possède néanmoins un défaut majeur lié à la non uniformité des positions possibles.

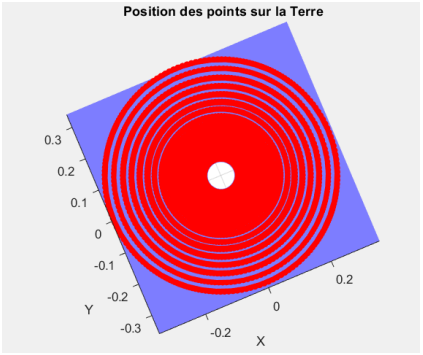


Figure 2.3 - Vue de haut de la calotte mettant en évidence la non uniformité des positions possibles du terminal avec la première méthode.

Nous détaillerons plus explicitement par la suite quelles en sont les conséquences. Cela nous amène ainsi à développer une seconde manière de construire cet ensemble.

2. Seconde méthode

Au lieu de créer dans un premier temps une matrice, puis de la projeter sur la calotte, nous allons cette fois choisir directement un point aléatoirement ayant une latitude et une longitude comprise dans la zone visible par le satellite. La longitude doit être prise dans l'intervalle $[-180°, +180°]$ et la latitude entre $[-\alpha_{max}, +\alpha_{max}]$ $où$ α_{max} correspond à l'angle limite à partir duquel le point pris n'est plus dans la zone de visibilité du satellite.

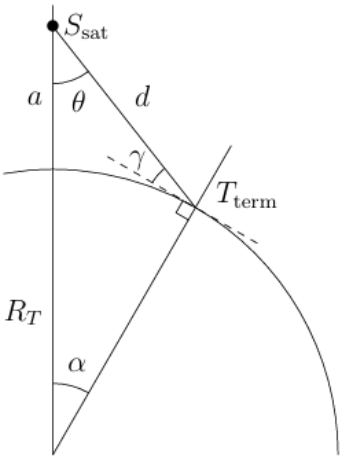


Figure 2.4 - Schéma des variables mises en jeu [1]

Par relations trigonométriques, on détermine α à partir de variables connues grâce à la formule:

$$\alpha_{max} = d_{max} \cdot \frac{sin(\theta_{3dB}/2)}{R_T}$$

Pour implémenter cette méthode avec le logiciel Matlab, nous avons privilégié l'utilisation de la fonction rand(N) qui renvoie un vecteur de taille N dont chacune des composantes correspond à un tirage suivant une loi uniforme sur [0, 1]. On effectue ensuite la transformation linéaire suivante pour obtenir un tirage uniforme sur les intervalles souhaités:

$longitude = -180 + 360 * rand(N)$

et $latitude = 90 - (-\alpha_{max} + 2 * \alpha_{max} * rand(N))$

En appliquant cette méthode pour le même nombre de points qu'à la partie précédente, on obtient la figure suivante :

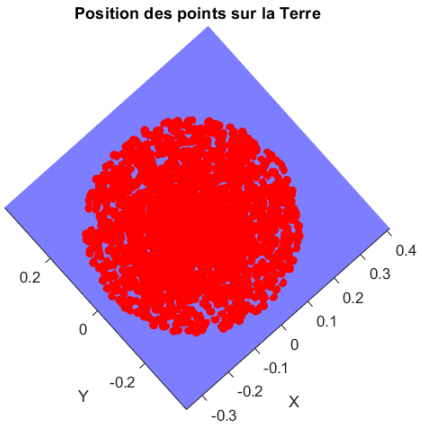


Figure 2.5 - Vue de haut de la calotte mettant en évidence une meilleure uniformité des positions possibles du terminal grâce à la seconde méthode.

Les positions possibles du terminal sont ici mieux réparties sur l'entièreté de la surface de couverture du satellite qu'avec la première méthode.

II.2 - Détermination des paramètres caractérisant le lien terminal satellite

Maintenant que nous avons la position du terminal, il est nécessaire de déterminer certains paramètres afin de pouvoir estimer le signal reçu par le satellite. Nous avons besoin de la distance entre le terminal et le satellite, du retard temporel, du décalage Doppler et du rapport Es/N0. La référence principale pour ce modèle est décrite dans le document [1] indiqué dans les références bibliographiques.

1. Distance au satellite et retard temporel

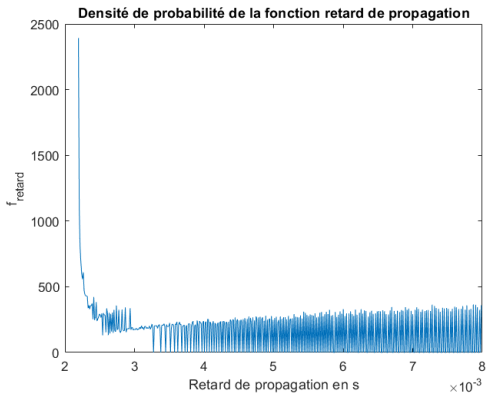
La distance entre le terminal et le satellite est calculé grâce au théorème d'Al-Kashi:

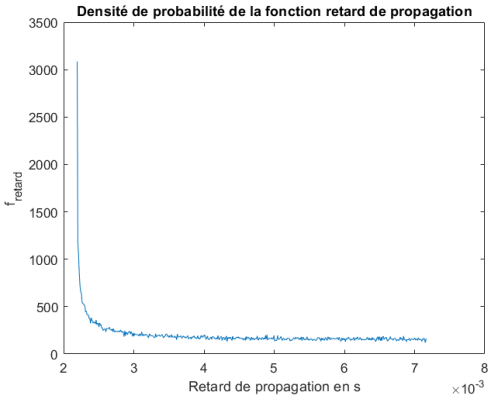
$$d(\alpha) = \sqrt{{R_T}^2 + (R_T + h)^2 - 2 \cdot R_T \cdot (R_T + h) \cdot cos(\alpha)} \; avec \; \alpha = \pi/2 - latitude$$

Une fois la distance calculée, on peut déterminer le retard temporel du signal dû au temps de propagation.

On a simplement $\tau(\alpha) = \frac{d(\alpha)}{c}$ avec c la vitesse de la lumière dans le vide.

On peut tracer les densités de probabilité de la fonction de retard de propagation trouvées en fonction de la méthode employée :





Figures 2.6 et 2.7 - A gauche, courbe faite avec la première méthode pour réaliser la grille, à droite courbe faite avec la seconde. Dans les deux cas, le tracé est fait avec un million de positions possibles et dix millions de réalisations.

On remarque bien comme annoncé précédemment que la méthode de construction de la grille a une forte influence sur le résultat obtenu. Ces deux courbes ont été tracées avec un ensemble de positions possibles de même cardinal (un million de positions possibles) et pour un même nombre de réalisation (dix millions). L'augmentation du nombre de réalisations n'a ici pas d'effet sur l'allure générale des courbes. On le gardera par la suite égal à dix fois le nombre de positions possibles. Par la suite, les tracés seront réalisés dans les mêmes conditions.

Les nombreuses discontinuités présentes dans la première courbe semblent être dûes à la non uniformité de la grille. En effet, par lecture graphique, la densité de probabilité de voir apparaître un bateau sur certaines zones est nulle. Cela correspond aux zones entières en forme de cercle complètement dépourvues de points sur lesquels peuvent apparaître des terminaux. Il est possible d'améliorer ces discontinuités en augmentant la taille de l'ensemble des positions possibles (cf figure 2.8). Intuitivement, cela correspond au cas où l'écart entre deux anneaux représentés figure 2.3 diminue.

Au contraire, la courbe construite avec la seconde méthode ne possède pas ces imperfections, et la fonction de retard de propagation est continue et semble suivre une loi exponentielle.

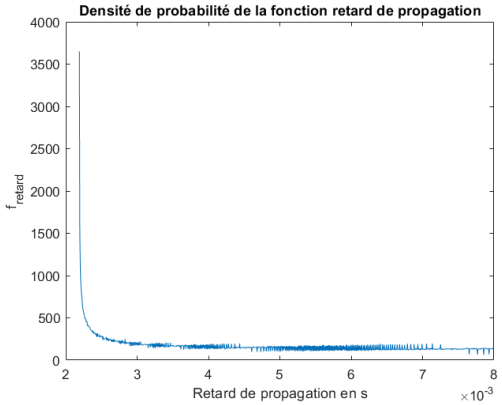


Figure 2.8 - Densité de probabilité de la fonction retard de propagation avec la première méthode lorsqu'il y a cent millions de positions possibles.

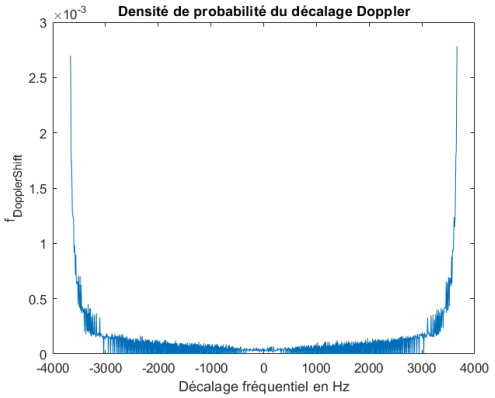
On observe lorsqu'on augmente le nombre de positions possibles que la courbe possède moins de discontinuités et ses discontinuités sont moins prononcées. En revanche, augmenter le nombre de positions possibles et le nombre de réalisations requiert un temps de calcul beaucoup plus important.

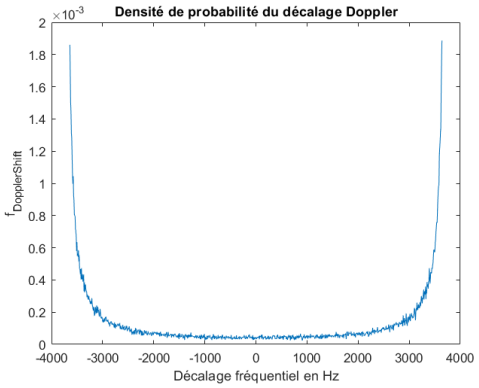
2. Décalage Doppler

Il est important dans le cadre des transmissions numériques d'étudier le décalage fréquentiel du signal. Dans un premier temps, ce décalage Doppler est utile dans notre simulateur pour estimer le signal reçu par le satellite. En pratique, le décalage Doppler cause aussi une dilatation de la bande de transmission qu'il faut prendre en compte.
Dans notre cas, le décalage Doppler s'écrit :

$$\Delta_f(t) = \frac{f_0}{c} \cdot \frac{\mathrm{d}d(t)}{\mathrm{d}t}$$

avec f_0 la fréquence d'émission du signal, $d(t)$ la variation de la vitesse radiale du satellite au cours du temps et c la vitesse de la lumière dans le vide.
L'expression de la vitesse radiale peut se déterminer géométriquement. Elle est explicitée dans [1] p33. Comme notre résultat est invariant en fonction du temps, on pose $t = 0$ et on obtient les courbes suivantes :





Figures 2.9 et 2.10 - A gauche, courbe faite avec la première méthode pour réaliser la grille, à droite avec la seconde.

On reconnaît bien la courbe caractéristique du décalage Doppler. On vérifie les valeurs trouvées “à la main” pour deux cas limites, lorsque le terminal est situé sur le pôle nord et lorsque le terminal est situé en limite de visibilité du satellite ($\alpha = \alpha_{max}$) et ces valeurs coïncident. On retrouve bien comme ce qui a été dit précédemment les défauts de la première version de la grille.

3. Rapport Es/N0

On définit le rapport Es/N0 comme le rapport entre l’énergie reçue d’un symbole et la densité spectrale du bruit. Il est homogène à un rapport signal à bruit en réception et nous permet de faire le bilan de liaison entre le terminal et le satellite. Dans notre cas son expression est la suivante, en unités linéaires :

$$\frac{E_s}{N_0} = \frac{PIRE \cdot (G/T)}{k_b \cdot R_c \cdot L_{tot}} \quad (2.1)$$

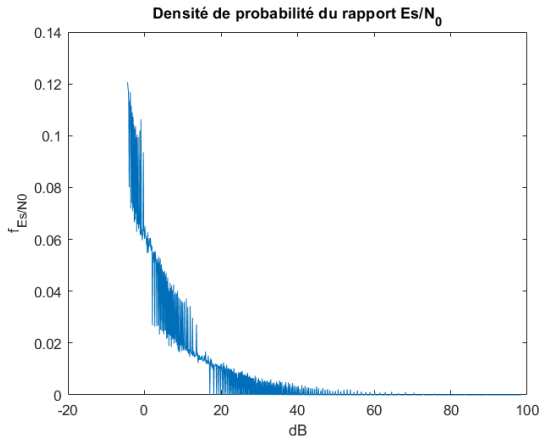
où la $PIRE$ est définie comme le produit de la puissance émise par le gain de l’antenne d’émission. G/T est un facteur de mérite relatif à l’antenne de réception du satellite. k_b est la constante de Boltzmann et R_c correspond au débit symbole.

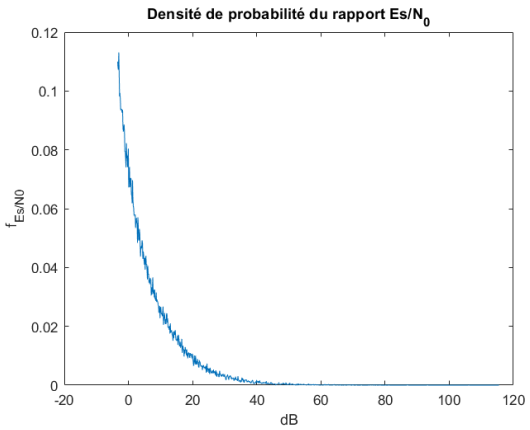
On définit L_{tot} dans l’équation (2.1) de la manière suivante :

$$L_{tot} = L_{pel} \cdot L_{ant,\,sat} \cdot L_{autres}$$

où $L_{pel} = (\frac{4\pi d}{\lambda})^2$ correspond aux pertes en espace libre, $L_{ant,\,sat} = 12(\frac{\theta}{\theta_{3dB}})^2$ modélise les pertes liées au dépointage de l’antenne et L_{autres} correspond aux pertes provenant d’autres sources.

On trace le rapport Es/N0 qu’on convertit en dB pour plus de lisibilité et on obtient les courbes suivantes :





Figures 2.11 et 2.12 - A gauche, courbe faite avec la première méthode pour réaliser la grille, à droite avec la seconde.

Il est plus difficile de déterminer analytiquement des valeurs pour déterminer la cohérence de ces courbes. Pour juger de la fiabilité du résultat obtenu, il a fallu comparer nos courbes avec celles obtenues dans la littérature, et notamment dans [1] qui a un problème similaire au nôtre.

Ainsi, nous avons construit l'environnement de simulation dans lequel va être transmis notre signal. Les paramètres qui ont été calculés sont fondamentaux pour pouvoir estimer le signal reçu par le satellite. Passons maintenant à la génération du signal.

III - Génération du signal

Rédacteurs: Théodore Diers, Lucas Perrin, Relecteur: Bastien Leseigneur

On souhaite envoyer un message sous la forme d'un signal, pour cela le message est d'abord mis sous la forme de 0 et de 1. Les contraintes du problème ont conduit à implémenter une norme de 168 bits d'information pour le message.
Pour que la transmission du message se fasse correctement, c'est-à-dire que l'on retrouve le message initial à la réception, il est nécessaire de prendre en compte certaines contraintes et d'y apporter des solutions.
En effet, le bruit et les interférences peuvent perturber l'intégrité du signal. C'est pourquoi il est nécessaire de vérifier cette intégrité à la réception du signal. La solution adoptée est l'ajout de 16 bits de CRC (Contrôle de redondance cyclique) en fin de message. Ce CRC est calculé à partir des bits du message, ce qui permet de vérifier à la réception que le crc reçu correspond au reste du message reçu. La correspondance entre ces deux éléments prouve l'intégrité du message. Dans certains systèmes, la technique d'ARQ (automatic repeat request) peut être mise en œuvre avec le CRC. Elle consiste à indiquer à l'émetteur que son message n'a pas été reçu correctement (émission d'un NACK) et l'émetteur ré-émet son message. Le CRC permet de valider si le message a été reçu correctement.

De plus, pour bien délimiter le message et en comprendre le contenu, il est nécessaire d'ajouter une séquence de bits (flags ou drapeaux) permettant au récepteur d'identifier le début et la fin du message. Cependant, les bits du message peuvent accidentellement reproduire cette séquence et ainsi perturber la bonne analyse des points de départ et de fin du message. C'est pourquoi il est nécessaire de modifier le message pour éviter cela, il s'agit de la technique de *bitstuffing* soit littéralement du bourrage de bits.
Enfin, une séquence de bits d'entraînement est ajoutée pour pouvoir repérer la présence d'un message AIS dans le signal reçu et d'identifier l'instant de début et la fréquence du signal AIS.
Une fois modifié ainsi, le signal numérique est modulé pour obtenir un signal analogique transmis sur le canal.
Le signal reçu est traité étape par étape pour extraire l'information binaire. Ci-dessous, les différentes fonctions sont décrites une à une avec la fonction de décodage associée.

III.1 - Génération aléatoire des bits

Le message doit être contenu dans une trame de 168 bits pour se conformer aux normes en vigueur imposées par le signal AIS. Ainsi, la première étape de notre modélisation fût la génération aléatoire d'une liste de 168 bits sur lesquels on a ensuite travaillé. Pour cela, nous avons utilisé la fonction *randi* de Matlab, qui crée une matrice ligne de la taille souhaitée. Ces 168 bits constituent le *payload* qui est la partie du message qui contient l'information.

```
random_bits = randi([0, 1], 1, 168);
```

Nous avons également testé cette fonction en créant un histogramme permettant de vérifier qu'avoir un 1 ou un 0 est équiprobable.

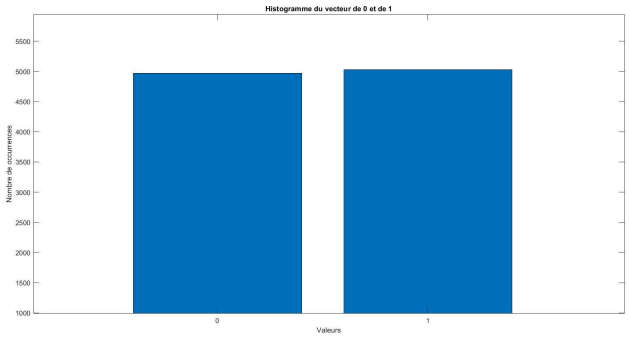


Figure 3.1 - Répartition des 0 et 1 dans la génération du payload

III.2 - Codage CRC

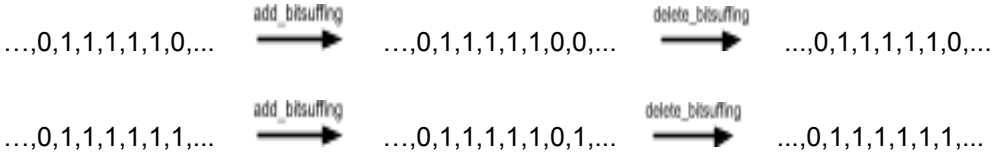
Nous avons ensuite implémenté les fonctions *add_crc(payload,generateur), test_crc(message_crc,generateur)* et *decodage_crc(message_with_crc)* qui respectivement ajoute le CRC, vérifie l'intégrité du message et retire le CRC. Le CRC (Contrôle de

Redondance Cyclique), complément à deux du FCS (Frame Checking Sequence) permet de vérifier que le message est correct. Le CRC est calculé en ajoutant 16 bits initialement à zéro à la fin du message, en inversant les 16 premiers et en calculant la division modulo 2 du message par un polynôme générateur (en binaire donc 10101 ~ $x^4 + x^2 + 1$). Les 16 bits obtenus en fin de message sont ensuite inversés et peuvent être concaténés au message initial [3]. Le processus peut être vu mathématiquement avec des équivalents polynômiaux (Annexe A de [3]).
Le test consiste à ajouter le complémentaire du CRC au message initial et de vérifier que le CRC de ce nouveau message est bien nul [3]. Si le CRC est bien nul, cela prouve que le CRC correspond bien au message, c'est-à-dire que l'on a une absence d'erreurs due à la transmission du message. Le CRC peut alors être retiré pour récupérer le message tel qu'envoyé.

Data	CRC

III.3 - Bitstuffing

Ensuite, des bits de bourrage sont ajoutés au message pour éviter une séquence de six 1, qui correspond à la séquence utilisée pour les flags de début et de fin de message (délimitation du message).
Pour cela, nous avons implémenté une fonction *add_bitstuffing(message)* qui modifie le message en ajoutant un 0 après chaque série de cinq 1. Simultanément, nous avons créé la fonction *delete_bitstuffung(message_with_bitstuffing)* qui permet de récupérer le message initial en supprimant les bits de bourrage ajoutés. Dès qu'une série de cinq 1 est détectée, le 0 qui suit est supprimé.
Nous avons également testé cette fonction en vérifiant qu'aucune occurrence de six 1 n'apparaît après le bitstuffing pour un nombre important de signaux. Pour cela nous avons implémenté une fonction *test_bitsuffing1* qui crée aléatoirement 1000 signaux et qui compte ceux avec plus de six 1 consécutifs, puis *test_bitstuffing2* qui crée aléatoirement 1000 signaux comportant tous au moins six 1 consécutifs. Le résultat montre qu'aucun signal en sortie n'en comporte et donc que notre bitstuffing fonctionne bien.



III.4 - Flags

Les flags de début et de fin sont alors ajoutés pour délimiter le message. Les flags sont la séquence [0,1,1,1,1,1,1,0].

Pour cela, on a créé la fonction *add_flag(message)*. De la même manière que pour le bitstuffing et le CRC, nous avons également créé une fonction *delete_flag(message_with_flags)* qui permet de retrouver le message initial.

Start Flag	Data	CRC	End Flag

III.5 - Training sequence

Enfin une séquence d'entraînement est ajoutée (Training Sequence). Celle-ci permet de repérer la présence d'un message AIS dans le signal reçu et d'identifier l'instant de début et la fréquence du signal AIS.
La training sequence est une liste de 24 bits alternant 0 et 1, commençant par 0 ou par 1 selon la convention choisie.
Nous avons ainsi créé les fonctions *add_training_sequence(message,convention)* et *delete_training_sequence(message_training_sequence)* qui respectivement l'ajoute ou la supprime.

Training Sequence	Start Flag	Data	CRC/FCS	End Flag

Structure finale du message envoyé- [2]

III.6 - Codage NRZI

Pour centrer le module de notre signal final en 0, il sera nécessaire de travailler avec non plus des bits de 0 et 1 mais des -1 et 1. C'est l'un des rôles du codage NRZI. Pour cela, on choisit un 1 comme première valeur du signal en sortie si la première valeur du signal est 1 ou -1 sinon, et on va considérer qu'à chaque 1 du signal initial, la valeur du signal en sortie change (1 passe à -1 et inversement).

Ainsi, par exemple, le codage NRZI amène la transformation suivante sur ce signal à 12 bits:

[1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1] → [1, 1, 1, -1, 1, 1, 1, -1, -1, -1, -1, 1]

Cette méthode de codage est particulièrement utile dans les systèmes AIS car elle permet de maintenir une synchronisation précise entre l'émetteur et le récepteur, réduisant les risques de désynchronisation causés par des séquences longues de bits identiques (comme une série de 0 ou de 1). De plus, le codage NRZI facilite la détection d'erreurs et améliore l'intégrité des données en rendant les transitions de signal plus détectables même en présence de bruit et de distorsion, conditions souvent rencontrées dans les communications par satellite.

Le **décodeur** fonctionne donc simplement de manière réciproque : on remplit une liste en connaissant donc le premier bit (0 si le signal après NRZI commence par -1 et 1 sinon), et on remplit la liste en comparant les symboles du signal NRZI un à un: si le prochain symbole diffère du précédent, on ajoute 1 à la liste, et 0 sinon; retrouvant ainsi finalement le signal initial.

Nous avons explicitement testé nos fonctions sur des signaux que nous avons créés, en vérifiant tout d'abord nous même que les signaux renvoyés étaient cohérents, puis nous avons généré 1000 signaux aléatoires auxquels nous avons appliqué *encode_NRZI* puis *decode_NRZI*, pour enfin regarder la différence du signal ainsi obtenu avec le signal initial. Nous avons ainsi eu 100% de listes nulles, confirmant bien que les deux fonctions sont réciproques et fonctionnent.

III.7 - Modulation GMSK

Ayant maintenant un signal composé de -1 et de 1, on va lui appliquer une dernière transformation : la modulation GMSK.
La modulation GMSK est une forme de modulation par déplacement de fréquence à phase continue. Elle est essentielle dans la transmission des signaux AIS car elle permet d'optimiser l'efficacité spectrale tout en minimisant les interférences et les distorsions du signal, ce qui est crucial pour les communications maritimes. La GMSK utilise un filtre gaussien pour lisser les transitions entre les bits, réduisant ainsi l'étalement spectral et les lobes secondaires. Cela permet au signal d'occuper une bande passante plus étroite, essentielle pour le partage efficace du spectre parmi de nombreux utilisateurs, en particulier dans des environnements où les interférences d'accès multiple sont courantes. En outre, la robustesse du GMSK face aux interférences et sa capacité à maintenir l'intégrité des données dans des environnements à faible rapport signal/bruit en font une technique de modulation idéale pour les applications AIS par satellite, où les signaux doivent souvent traverser de longues distances avec un minimum de dégradation.

Propriétés

Expression du signal en bande de base

$$s(t) = \sqrt{\frac{2E_s}{T_s}} \exp\left(i\phi(t)\right)$$

avec

$$\phi(t) = 2\pi h \sum_k a_k q(t - kT_s)$$

Figure 3.2 - Signal émis par le terminal [5]

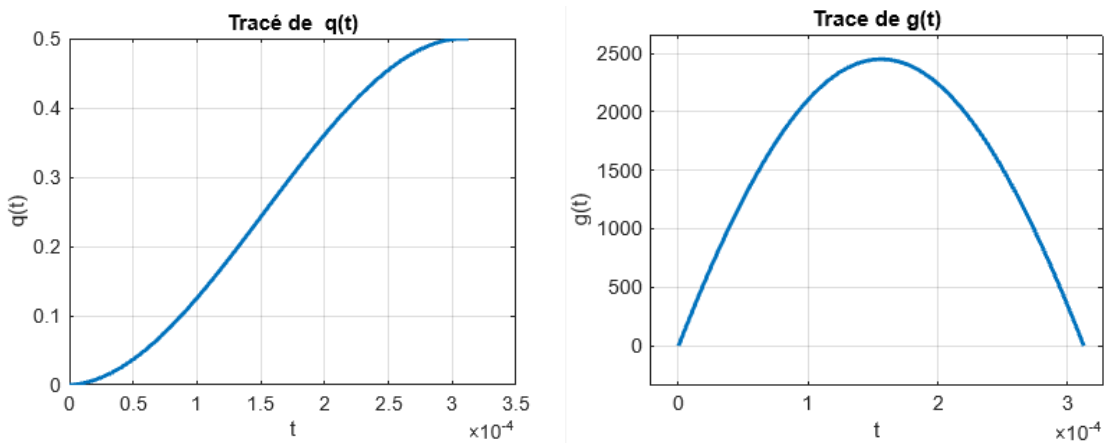


Figure 3.3 - Tracé des filtres g et q utilisés pour la GMSK

Ces tracés correspondent aux résultats attendus. A ce stade, le signal peut être envoyé sous la forme vue figure 3.3.

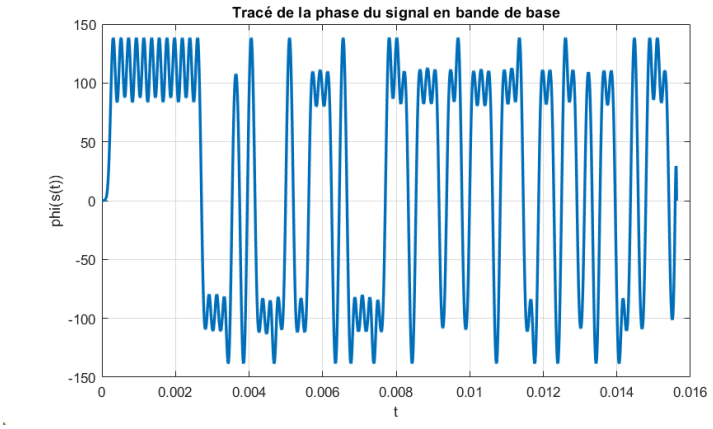


Figure 3.4 - Tracé de la phase du signal envoyé en bande de base

Nous avons décidé d'utiliser un treillis pour coder la modulation GMSK. En résumé, l'utilisation d'un treillis améliore la performance de démodulation et la correction d'erreurs en exploitant la structure temporelle et de phase de la modulation, optimisant ainsi la fiabilité et l'efficacité du système de communication. On garde en effet une mémoire essentielle, ce qui est crucial pour des techniques telles que le décodage Viterbi que nous allons utiliser.
Voici comment nous procédons:
les états du système sont représentés par trois variables : $\theta(u)$(4 réalisations possibles), $\alpha_1(u)$ et $\alpha_2(u)$ (respectivement 2 réalisations possibles {-1,1}).
Le nombre total d'états S à chaque instant est le produit des nombres de réalisations possibles de chaque variable : S=4×2×2=16.
Le treillis est un graphique où chaque colonne représente un instant de temps u et chaque nœud dans cette colonne représente un état $(\theta, \alpha_1, \alpha_2)$ Les branches entre les nœuds de

colonnes consécutives représentent les transitions possibles entre les états à des instants u et u+1.
Si à l'instant u l'état est $(\theta, \alpha_1, \alpha_2)$ et le symbole de données entrant est $d(u)$; le nouvel état à l'instant u+1 dépend de la manière dont $d(u)$ affecte θ, α_1 et α_2:
$\alpha_1(u+1)$ prend la valeur de $d(u)$ tandis que $\alpha_2(u+1)$ prend la valeur de $\alpha_1(u)$.
Pour θ, si $d(u)$ vaut 1, on ajoute +π/2 et si $d(u)$ vaut -1, on ajoute -π/2.

Exemple de transition:

Temps u Temps u+1

(0, -1,1) ⟶ $d(u) = 1$ ⟶ ($\frac{\pi}{2}$, 1,-1)

(0, -1,-1) ⟶ $d(u) = -1$ ⟶ ($-\frac{\pi}{2}$,-1, -1)

Nous avons donc dans un premier temps déterminé le train d'éléments binaires à envoyer, que nous avons ensuite codés par codage NRZI ce qui nous a permis d'obtenir un signal composé uniquement de -1 et de 1. Cela nous a ensuite permis de moduler le signal à envoyer par modulation GMSK. L'information à transmettre est alors contenue dans la phase de ce signal. Enfin, nous avons estimé le signal en bande de base émis en sortie du terminal. Nous nous intéresserons maintenant à la réception et au décodage du signal lorsqu'il est arrivé au satellite.

IV - Réception et décodage

Rédacteurs: Bastien Leseigneur, Lucas Perrin, Relecteur: Théodore Diers

IV.1 - Estimation du signal reçu

Le signal reçu par le satellite est somme du signal émis après GMSK et d'un bruit ajouté par la transmission. On modélise ce bruit par un bruit additif blanc gaussien. Le signal reçu s'écrit explicitement après échantillonnage:

$$r_n = \sqrt{\frac{1}{F}\left(\frac{E_s}{N_0}\right)} e^{i\phi((n-\Delta)\frac{T_z}{F}} e^{i(2\pi f_d \frac{nT_s}{F})} + w_n$$

où le ϕ correspond à la phase calculée précédemment, et où la seconde exponentielle est calculée en partie à partir des paramètres explicité dans la génération de l'environnement de transmission.

Le bruit w_n est modélisé par une suite de variables aléatoires complexes indépendantes et identiquement distribuées selon une loi Gaussienne circulaire symétrique et de variance unité. On peut l'écrire sur Matlab sous la forme du vecteur:
w = 1/$\sqrt{2}$. (randn(1, N) + i.randn(1, N)) où randn est une fonction Matlab qui renvoie un vecteur de taille N dont les éléments sont un tirage d'une loi normale centrée réduite.

IV.2 - Algorithme de Viterbi

La modulation GMSK est modélisée sous forme de treillis, où chaque état du treillis représente une possible phase du signal à un instant donné. Les transitions entre les états du treillis correspondent aux symboles transmis et tiennent compte de la continuité de phase imposée par la GMSK.
Pour chaque échantillon de signal reçu, l'algorithme de Viterbi calcule une métrique de similarité entre le signal reçu et les signaux attendus pour chaque transition possible dans le treillis.
Chaque état du treillis conserve une métrique de chemin cumulative, qui représente la somme des métriques de similarité des transitions successives menant à cet état.
Pour chaque état, l'algorithme conserve non seulement la corrélation la plus élevée mais aussi une trace du chemin emprunté pour arriver à cet état.
Une fois tous les échantillons reçus et traités, l'algorithme de Viterbi effectue un retour arrière à partir de l'état final le plus probable pour reconstituer la séquence de symboles la plus probable transmise.

Malheureusement, nous n'avons pas eu le temps de finir l'implémentation de l'algorithme de Viterbi. Une première version est en développement mais des problèmes persistent et le temps manque: il est donc impossible de tester le code de bout en bout et de tester complètement le modèle.
A MODIFIER MTN QU'ON A REUSSI

IV.3 - Décodage du signal

Une fois l'algorithme de Viterbi effectué, nous retrouvons une séquence de symbole qu'il faut décoder. Lors de la génération du signal vu dans la partie précédente, nous avons fait en sorte d'associer à chaque fonction (*add_training_sequence* par exemple) sa fonction réciproque. Ces fonctions ont déjà été explicitées et détaillées dans la partie précédente (III).

On applique donc successivement au signal le décodage NRZI permettant de convertir la chaîne de -1 et 1 en un train d'éléments binaires. On enlève la séquence d'entraînement (ou training sequence) et on enlève les drapeaux en début et fin de signal. Il faut ensuite enlever les bits de bourrage pour se retrouver avec un signal uniquement composé du payload et du CRC.

On applique le décodage CRC. A ce moment, on effectue un test pour vérifier que le signal contenu dans le payload n'a pas été altéré. Ce test consiste à calculer le CRC du message complété par le complémentaire du CRC et à vérifier que l'on trouve une liste de 16 zéros.

Si ce test est validé, le message est identique à celui envoyé et on peut enlever le CRC et récupérer le cœur du signal envoyé contenant les informations que l'on voulait transmettre. Sinon le message doit être envoyé à nouveau.

IV.4 - Performances du détecteur

Compte tenu que nous n'avons pas réalisé l'algorithme de Viterbi, nous n'avons pas pu tester les performances du détecteur. Cette partie permettait de valider ou non notre modèle. La première partie de test de performances du détecteur consiste à envoyer un signal sans le bruit. On considère dans ce cas qu'à la réception $w_n = 0$. Notre détecteur aurait dû dans ce cas avoir 100% de détection.

Dans le cahier des charges que nous avions établi, nous avions mentionné une contrainte sur la performance en détection en présence de bruit : Détection de 99% des messages pour une SNR (Rapport Signal sur Bruit) de 19 dB. Nous aurions pu alors réaliser un grand nombre de tests dans ces conditions et vérifier, ou non si nous validions cette contrainte.

A MODIFIER MTN QU'ON A REUSSI

V - Conclusion

Rédacteurs: Bastien Leseigneur, Lucas Perrin, Relecteur: Théodore Diers

Ce projet, intitulé "Détection des communications AIS par satellite en présence d'interférences d'accès multiple", avait pour objectif de développer un simulateur robuste de communications AIS entre un terminal situé sur Terre et un satellite. Il devait être capable de construire un environnement adéquat à ce type de transmission numérique, de générer et de moduler un signal AIS respectant le format imposé ainsi que de revoir, décoder et démoduler ce signal même en présence de bruit. Nous n'avons malheureusement que partiellement complété et validé ce simulateur. Nous avons validé la simulation de l'environnement, la génération du signal AIS, l'encodage et la modulation du signal ainsi que les fonctions réciproques de décodage. Nous n'avons pas pu effectuer la mise en place de l'algorithme de Viterbi ce qui nous a pénalisés dans l'étude des performances du détecteur et de l'estimateur des paramètres de décalage temporels et fréquentiels. Pendant toute la durée du projet, nous avons fait en sorte de réaliser un programme le plus modulable possible pour faciliter sa reprise et sa modification s'il venait à être réutilisé.
Si nous avions eu plus de temps, il aurait été intéressant d'observer l'influence de l'émission en simultané de plusieurs terminaux sur notre détecteur. En effet, l'origine du problème qui a donné naissance à ce projet est la présence d'interférence d'accès multiples liées à la collision entre signaux provenant de différents terminaux.

VI - Glossaire

AIS : Automatic Identification System

TDMA : Time-Division Multiple Access

CRC : Contrôle de Redondance Cyclique

Découpage SOTDMA : Découpage Self-Organizing TDMA

GMSK : Gaussian Minimum-Shift Keying

PIRE : Puissance Isotrope Rayonnée Équivalente

NRZI : Non Return To Zero Inverted

FCS : Frame Checking Sequence

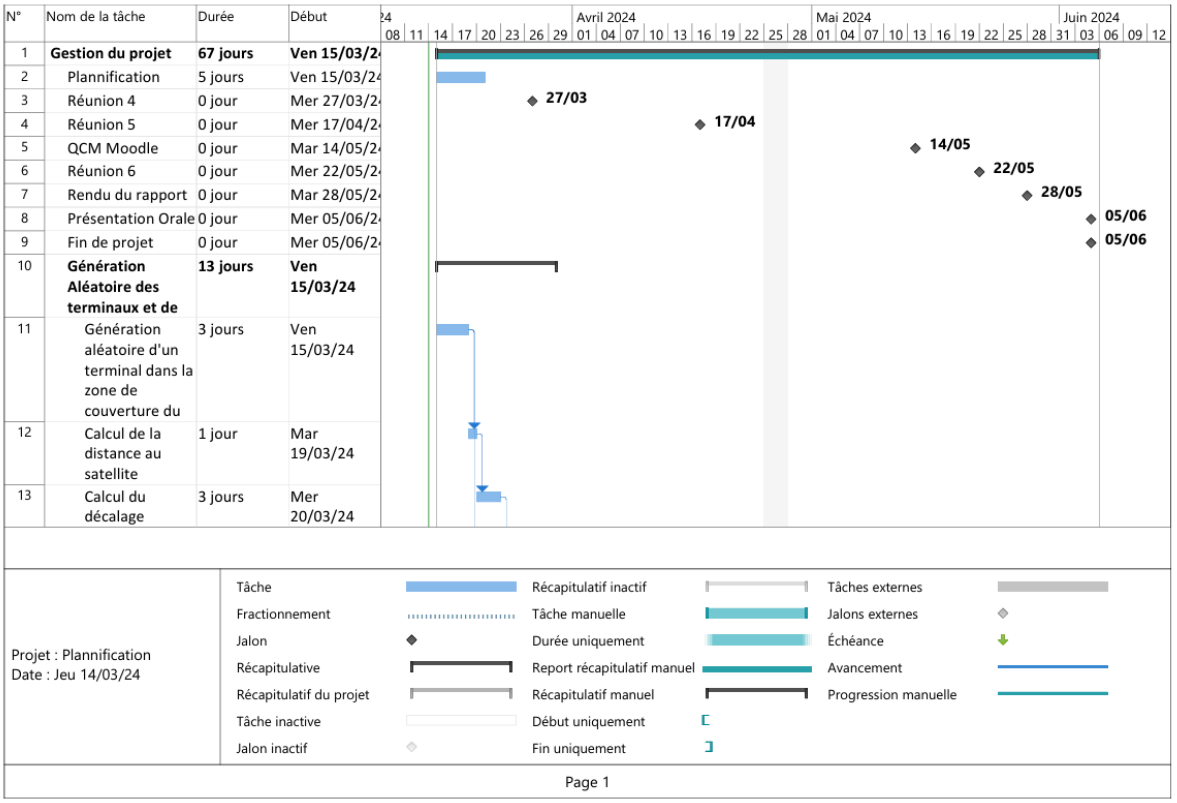
VII - Annexes

Rédacteur: Bastien Leseigneur, Relecteurs: Théodore Diers, Lucas Perrin

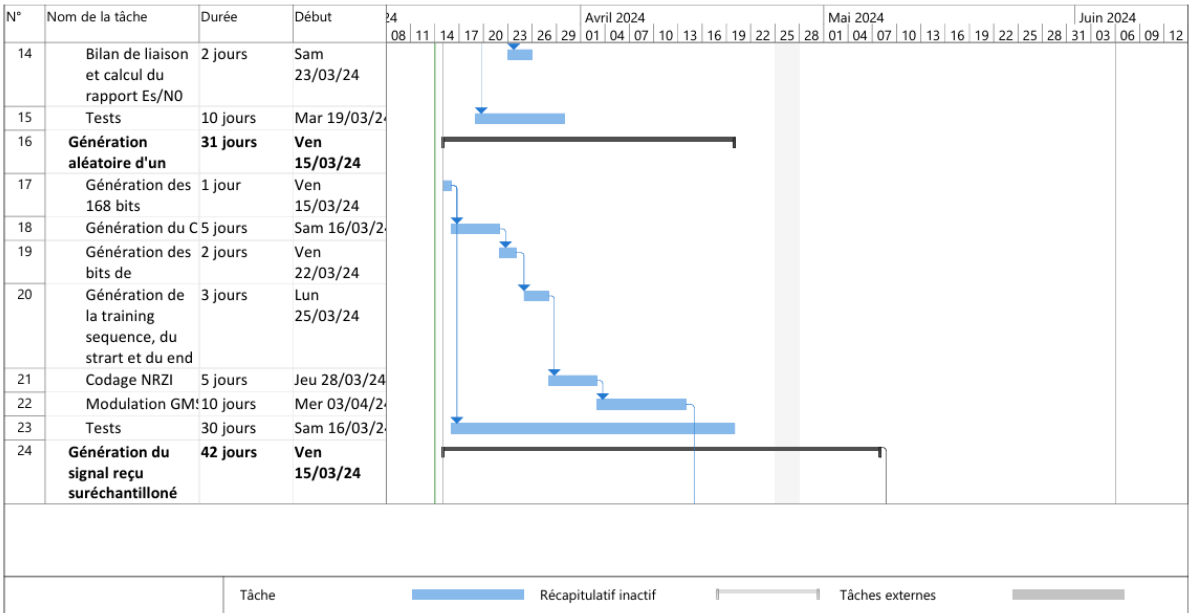
Boltzmann's constant k_B	
Distance between the Satellite and the Terminal $d(\alpha)$	
Doppler shift $S_D(\alpha)$	
Doppler rate $R_D(\alpha)$	
f_0	162 MHz
Modulation rate R_s	9600 bauds
Satellite altitude a	656.5 km
Satellite speed v_{sat}	7531 m/s
3 dB Beamwidth θ_{3dB}	130 degrees
G/T (Satellite)	-30 dBi/K
P_T (Terminal)	12 Watts
Antenna gain G_A (Terminal)	2.2 dBi
Propagation loss $L_P(\alpha)$	$20 \log_{10}\left(\frac{4\pi d(\alpha)}{\lambda}\right)$ dB
Antenna pointing loss (satellite) $L_D(\alpha)$	$12\left(\frac{\theta(\alpha)}{\theta_{3dB}}\right)$ dB
Other losses L_O	2 dB

Tableau 1 - Paramètres utilisés lors de la simulation

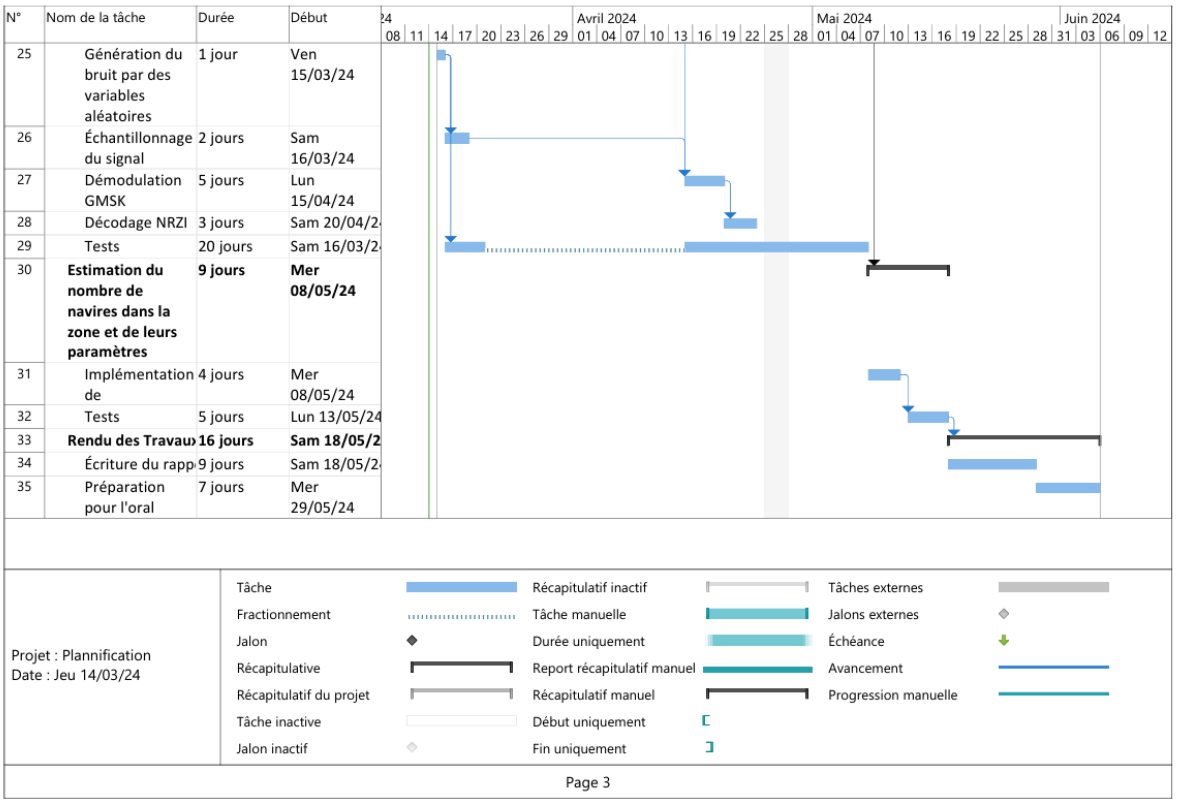
VII.1 - Comparaison des plannings réels et prévisionnels



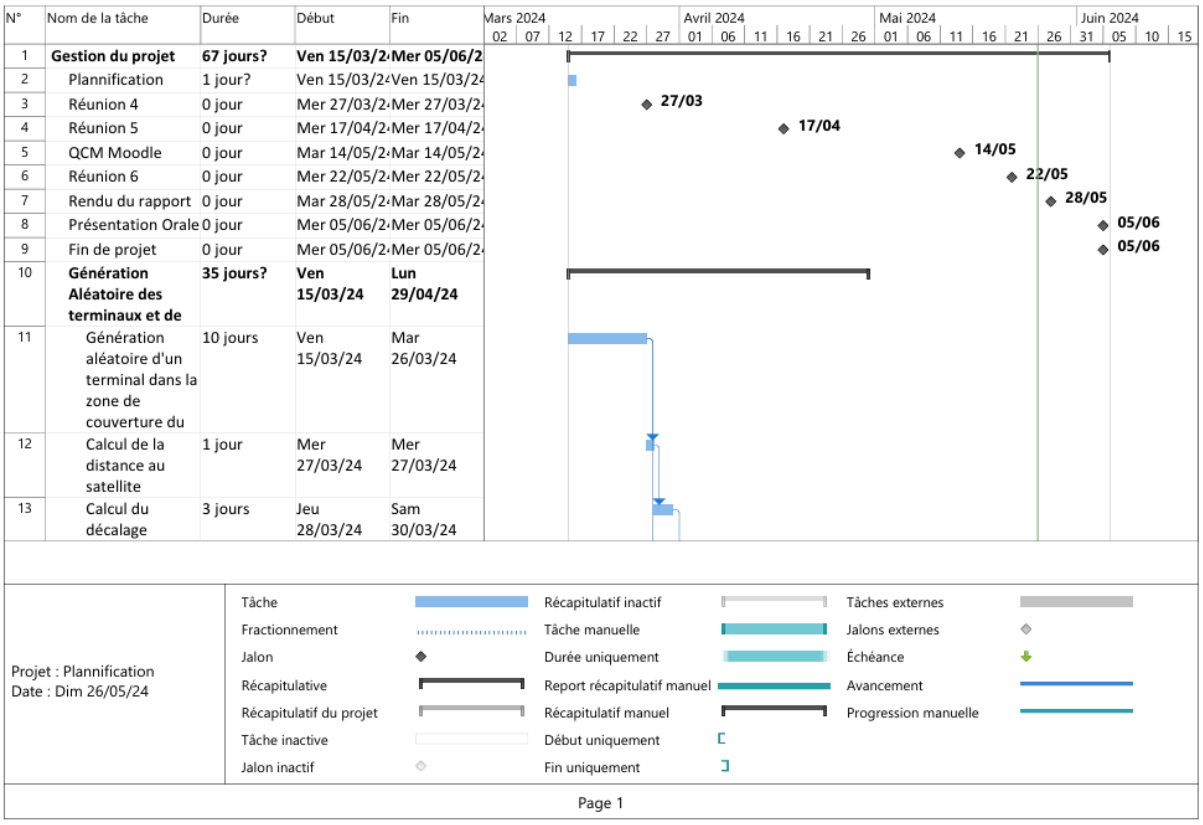
N°	Nom de la tâche	Durée	Début
1	**Gestion du projet**	**67 jours**	**Ven 15/03/24**
2	Plannification	5 jours	Ven 15/03/24
3	Réunion 4	0 jour	Mer 27/03/24
4	Réunion 5	0 jour	Mer 17/04/24
5	QCM Moodle	0 jour	Mar 14/05/24
6	Réunion 6	0 jour	Mer 22/05/24
7	Rendu du rapport	0 jour	Mar 28/05/24
8	Présentation Orale	0 jour	Mer 05/06/24
9	Fin de projet	0 jour	Mer 05/06/24
10	**Génération Aléatoire des terminaux et de**	**13 jours**	**Ven 15/03/24**
11	Génération aléatoire d'un terminal dans la zone de couverture du	3 jours	Ven 15/03/24
12	Calcul de la distance au satellite	1 jour	Mar 19/03/24
13	Calcul du décalage	3 jours	Mer 20/03/24



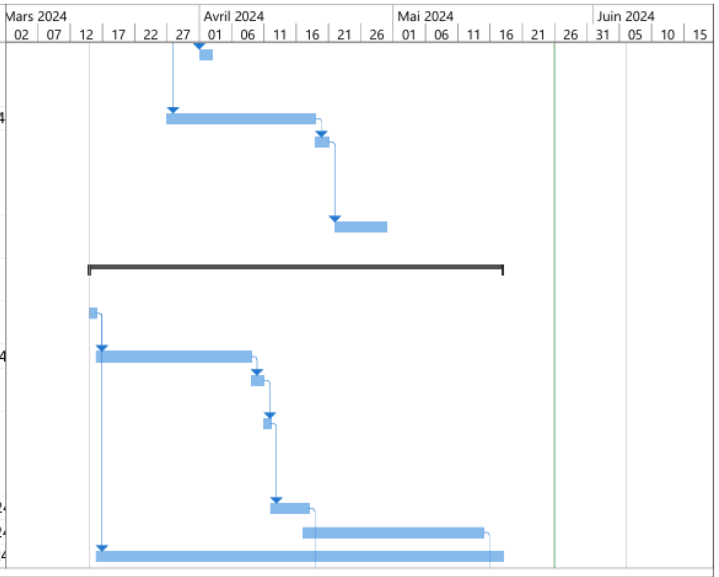
N°	Nom de la tâche	Durée	Début
14	Bilan de liaison et calcul du rapport Es/N0	2 jours	Sam 23/03/24
15	Tests	10 jours	Mar 19/03/24
16	**Génération aléatoire d'un**	**31 jours**	**Ven 15/03/24**
17	Génération des 168 bits	1 jour	Ven 15/03/24
18	Génération du C	5 jours	Sam 16/03/24
19	Génération des bits de	2 jours	Ven 22/03/24
20	Génération de la training sequence, du strart et du end	3 jours	Lun 25/03/24
21	Codage NRZI	5 jours	Jeu 28/03/24
22	Modulation GM	10 jours	Mer 03/04/24
23	Tests	30 jours	Sam 16/03/24
24	**Génération du signal reçu suréchantilloné**	**42 jours**	**Ven 15/03/24**



Planning 1 - Planning prévisionnel du déroulé du projet



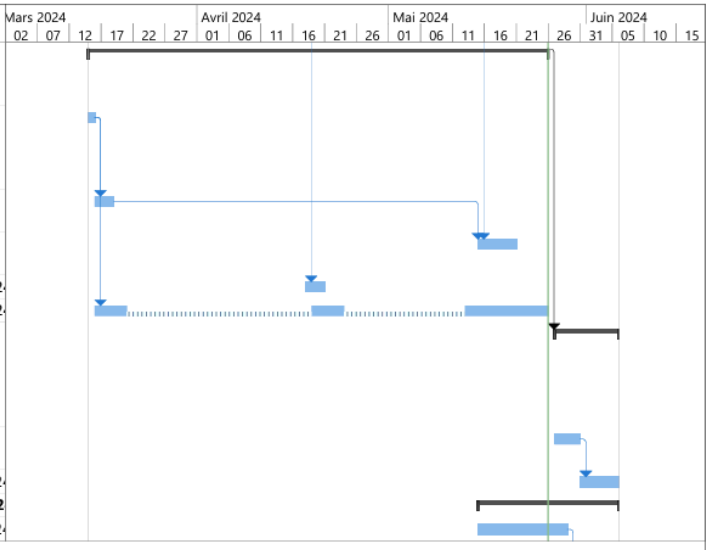
N°	Nom de la tâche	Durée	Début	Fin
14	Bilan de liaison et calcul du rapport Es/N0	2 jours	Lun 01/04/24	Mar 02/04/24
15	Tests	20 jours	Mer 27/03/2	Jeu 18/04/24
16	2eme méthode de génération aléatoire d'un terminal	2 jours?	Ven 19/04/24	Sam 20/04/24
17	2eme phase de tests	3 jours	Lun 22/04/24	Lun 29/04/24
18	**Génération aléatoire d'un**	**51 jours**	**Ven 15/03/24**	**Ven 17/05/24**
19	Génération des 168 bits	1 jour	Ven 15/03/24	Ven 15/03/24
20	Génération du C	20 jours	Sam 16/03/2	Lun 08/04/24
21	Génération des bits de	2 jours	Mar 09/04/24	Mer 10/04/24
22	Génération de la training sequence, du strart et du end	1 jour	Jeu 11/04/24	Jeu 11/04/24
23	Codage NRZI	5 jours	Ven 12/04/2	Mer 17/04/2
24	Modulation GM	20 jours	Mer 17/04/2	Mar 14/05/2
25	Tests	50 jours	Sam 16/03/2	Ven 17/05/24



Projet : Plannification
Date : Dim 26/05/24

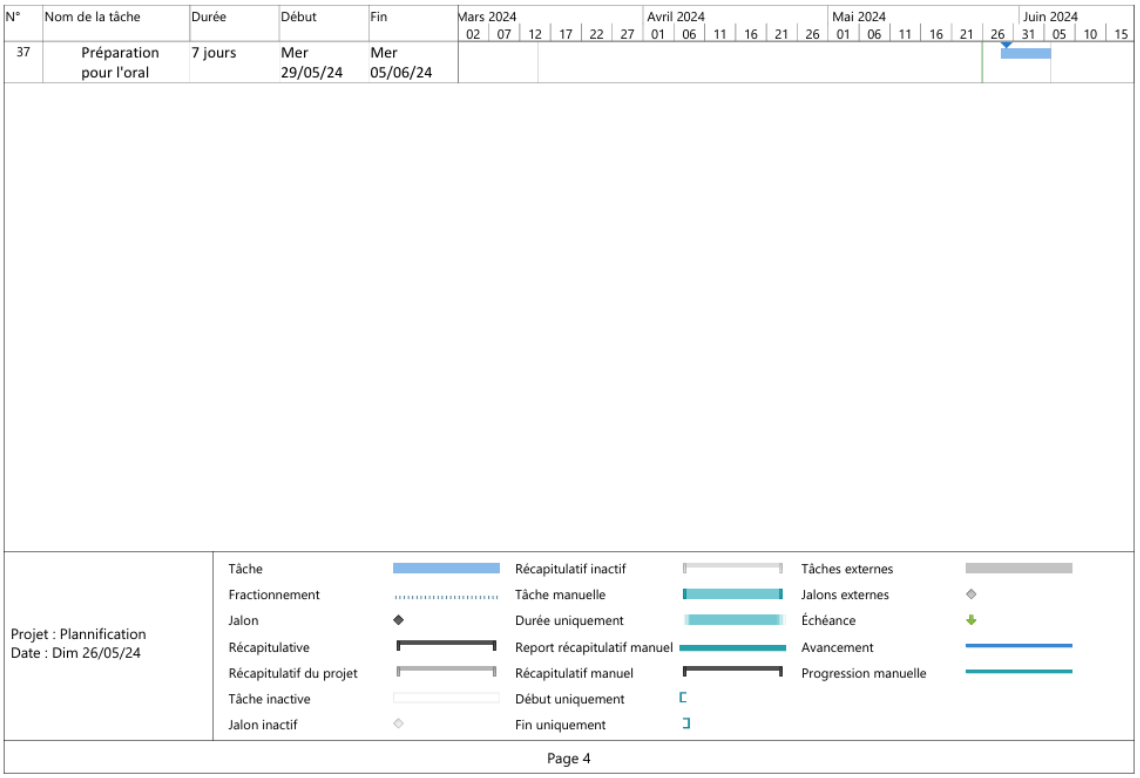
Tâche
Fractionnement
Jalon
Récapitulative
Récapitulatif du projet
Tâche inactive
Jalon inactif
Récapitulatif inactif
Tâche manuelle
Durée uniquement
Report récapitulatif manuel
Récapitulatif manuel
Début uniquement
Fin uniquement
Tâches externes
Jalons externes
Échéance
Avancement
Progression manuelle

N°	Nom de la tâche	Durée	Début	Fin
26	**Génération du signal reçu suréchantilloné**	**58 jours**	**Ven 15/03/24**	**Sam 25/05/24**
27	Génération du bruit par des variables aléatoires	1 jour	Ven 15/03/24	Ven 15/03/24
28	Échantillonnage du signal	2 jours	Sam 16/03/24	Lun 18/03/24
29	Démodulation GMSK	5 jours	Mer 15/05/24	Lun 20/05/24
30	Décodage NRZI	3 jours	Jeu 18/04/24	Sam 20/04/2
31	Tests	20 jours	Sam 16/03/2	Sam 25/05/2
32	**Estimation du nombre de navires dans la zone et de leurs paramètres**	**9 jours**	**Lun 27/05/24**	**Mer 05/06/24**
33	Implémentation de	4 jours	Lun 27/05/24	Jeu 30/05/24
34	Tests	5 jours	Ven 31/05/2	Mer 05/06/2
35	**Rendu des Travau**	**19 jours**	**Mer 15/05/2**	**Mer 05/06/2**
36	Écriture du rapp	12 jours	Mer 15/05/2	Mar 28/05/2



Projet : Plannification
Date : Dim 26/05/24

Tâche
Fractionnement
Jalon
Récapitulative
Récapitulatif du projet
Tâche inactive
Jalon inactif
Récapitulatif inactif
Tâche manuelle
Durée uniquement
Report récapitulatif manuel
Récapitulatif manuel
Début uniquement
Fin uniquement
Tâches externes
Jalons externes
Échéance
Avancement
Progression manuelle



Planning 2 - Planning réel du déroulé du projet

Plusieurs écarts sont à noter entre le planning prévisionnel et le planning réel. Le premier à mentionner se situe dans la première partie nommée génération aléatoire des terminaux. Lors de la phase de tests nous nous sommes rendus compte que la méthode implémentée pour générer les terminaux (cf II.1) n'était pas complètement satisfaisante. Il a fallu implémenter une nouvelle méthode, puis la tester afin de valider cette fonctionnalité. Cela nous a donc pris plus de temps que prévu.

Une deuxième différence notable est la Génération du CRC dans la partie Génération aléatoire d'un signal AIS. La durée réelle est bien plus longue que la durée prévisionnelle. Cela est dû au fait que nous avons eu beaucoup du mal à implanter la méthode de division polynomiale sur Matlab (cf III.2). Il n'y a aussi que peu de littérature disponible en ligne donnant des exemples concrets de valeurs exactes de CRC en fonction du polynôme générateur trouvé. La partie de tests a donc aussi pris beaucoup plus de temps pour valider que ce que nous avions anticipé.

Enfin, la partie modulation GMSK a aussi pris plus de temps que prévu nous ralentissant dans l'avancée du projet. Nous avons eu des problèmes d'implémentation du calcul des filtres g et q (cf III.7) sur Matlab à cause du calcul intégral mis en jeu. Les fonctions de calcul direct d'intégrale sur Matlab ne donnant pas un résultat cohérent, nous avons dû essayer d'autres méthodes pour calculer ce filtre, en essayant par exemple de mettre en place une méthode de Newton.

Tous ces ralentissements dans l'avancement du projet ont fait que nous n'avons pas pu le mener jusqu'au bout comme nous l'aurions souhaité.

VIII - Bibliographie

[1] Sylvain Cluzel. « Système M2M/IoT par satellite pour l'hybridation d'un réseau NB-IoT via une constellation LEO » *Mémoire de thèse, Doctorat de l'Université de Toulouse délivré par ISAE*, 2019. p 26 à 36

[2] Cervera, Miguel A., Ginesi, Alberto. « On the performance analysis of a satellite-based AIS system ». *Signal Processing for Space Communications*, 2008

[3] « Information technology — Telecommunications and information exchange between systems — High-level data link control (HDLC) procedures ». 2002, édition 3

[4] Burzigotti, Paolo, Ginesi, Alberto, Colavolpe, Giulio. « Advanced Receiver Design for Satellite-Based AIS Signal Detection ». *5th Advanced Satellite Multimedia Systems Conference and the 11th Signal Processing for Space Communications Workshop*, 2010.

[5] Karine Amis. « Modulations GMSK - Systèmes AIS - Modèles ». *IMT Atlantique*, 2023